

تعلم البرمجة مع Claude Code

من الفهم والتجربة إلى بناء المشاريع
خطوة بخطوة بمساعدة Claude Code

للمبتدئ والطموح

فكرة
اليوم
مشروع
الغد

أَتَعَلَّم
أَجْرِب
أَبْنِي
مُسْتَقْبَلِي



أساسيات البرمجة
بناء المشاريع
Claude Code
مستقبل أفضل

☐ أتعلم
☐ أطبق
☐ أبني مشروعاً
☐ أحقق هدفي

كود
أفضل
لحياة
أجمل

برومبتات عربية

جميع الحقوق محفوظة لقناة برومبتات عربية.

هذا المنتج مخصص للاستخدام الشخصي للمشتري. لا يجوز إعادة نشره أو بيعه أو توزيعه أو اقتباس أجزاء منه لأغراض تجارية دون إذن كتابي من المالك.

القناة الرسمية: <https://t.me/PromptsArabic>

اسم المنتج	تعلم البرمجة مع Claude Code
الإصدار	الإصدار الأول
اللغة	العربية
المقاس	A4 عمودي — مهيأ للقراءة على الهاتف
تمت مراجعة معلومات وميزات 6 سبتمبر 2026 هذا الدليل حتى تاريخ	
مصادر التحقق	(code.claude.com/docs (permission modes, memory, sessions و python.org (الإصدار المستقر)

ملاحظة مهمة عن تاريخ المحتوى

منصات الذكاء الاصطناعي تتغير بسرعة. كل ميزة مذكورة هنا تم التحقق منها من المصدر الرسمي بتاريخ 6 سبتمبر 2026. إذا مر وقت طويل على هذا التاريخ، راجع الصفحة الرسمية للميزة قبل الاعتماد عليها في عمل حساس.

عن هذا الدليل

أسهل شيء أن تطلب من أداة ذكاء اصطناعي أن تكتب لك برنامجا. وأسوأ نتيجة أن تحصل على برنامج يعمل ولا تفهم منه سطورا. هذا الدليل يعكس المعادلة: كل درس يعطيك مفهوما، ثم مهمة تكتبها بنفسك، ثم برومبت مراجعة يجعل Claude Code يصحح محاولتك ويشرح سبب الخطأ دون أن يعطيك الحل، ثم اختبارا، ثم تحسينا. تتعلم البرمجة فعلا، وتتعلم في الوقت نفسه كيف تستخدم أداة وكيلة بطريقة لا تفقدك المهارة.

لمن هذا الدليل

- من يبدأ البرمجة من الصفر ولا يريد أن يصبح ناسخا للأكواد
- من تعلم أساسيات متفرقة ويريد ترتيبها في منهج
- الطلاب الذين يستخدمون الذكاء الاصطناعي في الواجبات ويخشون أن يفقدوا الفهم
- من يعرف لغة ويريد طريقة منظمة لتعلم لغة ثانية

ماذا ستتعلم

- منهج Learn ثم Try ثم Review ثم Fix ثم Build وكيف تطبقه على أي موضوع
- كيف تضبط Claude Code في وضع المعلم بحيث يرفض أن يعطيك الحل
- الأساسيات: قراءة الكود، والمتغيرات، والأنواع، والشروط، والحلقات، والدوال
- هياكل البيانات والبرمجة الكائنية والاستثناءات والملفات والوحدات
- الواجهات البرمجية وقواعد البيانات و Git والاختبارات والتصحيح
- أساسيات الأمان والكود النظيف
- ثلاثة مشاريع متدرجة تبني فيها شيئا يعمل

قبل أن تبدأ

المتطلبات

- Python مثبت — الإصدار المستقر وقت مراجعة هذا الدليل هو 3.14
- محرر نصوص، ويفضل VS Code
- Claude Code مثبت ويعمل — راجع دليل Claude Code في هذه السلسلة
- مجلد فارغ للتمارين، داخل مستودع Git
- الاستعداد لكتابة كود خاطئ أولاً. هذا شرط التعلم لا عائق له

كيف تستخدم هذا الدليل

- درس واحد في الجلسة الواحدة. لا تقرأ خمسة دروس دفعة واحدة
- اكتب محاولتك قبل أن ترسل برومبت المراجعة — هذه هي القاعدة الوحيدة غير القابلة للتفاوض
- إذا صح لك Claude خطأ، أعد كتابة الحل من الصفر لا تعديل السطر فقط
- لا تنتقل لدرس جديد قبل أن تجيب على أسئلة اختبار فهمك دون النظر
- المشاريع المصغرة ليست اختيارية — هي المكان الذي يثبت فيه ما تعلمته

طريقة القراءة الموصى بها

لا تقرأ الدليل كله دفعة واحدة. اقرأ فصلاً واحداً، ثم نفذ تمرينه على مهمة حقيقية من عملك، ثم انتقل. أربعة فصول مطبقة أفضل من أربعة عشر فصلاً مقروءاً.

8	لماذا يفشل معظم من يتعلم البرمجة بالذكاء الاصطناعي
8	الوهم الذي يقتل التعلم
8	منهج هذا الدليل
9	قاعدة العشرين دقيقة
9	ما لن يفعله هذا الدليل
10	اضبط Claude Code في وضع المعلم
10	الخطوة الأولى: مجلد التعلم
10	الخطوة الثانية: ملف CLAUDE.md التعليمي
11	الخطوة الثالثة: وضع الأذونات المناسب للتعلم
12	الخطوة الرابعة: اختبر أن الضبط يعمل
12	برومبتات ستستخدمها في كل درس
14	قراءة الكود قبل كتابته
14	اقرأ ثم توقع ثم شغل
16	اسأل عن الكود لا عن الحل
18	المتغيرات وأنواع البيانات
18	الأنواع الأساسية وما تسمح به
20	تسمية المتغيرات
22	الشروط واتخاذ القرار
22	if و elif و else
24	الشروط المركبة والقيم الصادقة

● الحلقات والتكرار

26

26

for و while ومتى تستخدم كلا منهما

28

التكرار على الهياكل وفهم القوائم

30

● الدوال

30

دالة واحدة، مسؤولية واحدة

32

الوسائط والقيم الافتراضية والفخ الشهير

34

● هياكل البيانات

34

القائمة والمجموعة والقاموس والصف

37

● البرمجة الكائنية

37

من قاموس إلى صنف

38

الوراثة والتركيب

41

● الاستثناءات ومعالجة الأخطاء

41

التقاط ما تتوقعه فقط

43

● الملفات والوحدات والحزم

43

قراءة الملفات وكتابتها بأمان

44

الوحدات والحزم واستيرادها

47

● الواجهات البرمجية APIs

47

طلب HTTP ومعالجة استجابته

49

● قواعد البيانات

49

SQLite: قاعدة بيانات في ملف

51

● Git: تاريخ عملك

51	الدورة اليومية
53	● الاختبارات
53	اختبار الوحدة
55	● التصحيح المنهجي
55	من الخطأ إلى السبب في خمس خطوات
58	● أساسيات الأمان والكود النظيف
58	الخطأ الأول: الأسرار في الكود
58	الخطأ الثاني: الثقة في المدخلات
59	الخطأ الثالث: رسائل خطأ تكشف الداخل
59	الكود النظيف في خمس قواعد
60	● ثلاثة مشاريع متدرجة
60	المشروع الأول: مدير مهام في الطرفية
60	المشروع الثاني: محلل ملفات نصية
60	المشروع الثالث: خدمة صغيرة بقاعدة بيانات
61	كيف تدير كل مشروع

لماذا يفشل معظم من يتعلم البرمجة بالذكاء الاصطناعي

المشكلة ليست في الأداة. المشكلة أن الأداة تجيب بسرعة، والتعلم يحتاج تعثراً. عندما تحصل على الحل قبل أن تحاول، يشعر دماغك بأنه فهم، بينما لم يبن شيئاً.

الوهم الذي يقتل التعلم

اقرأ حلاً صحيحاً وستقول: كنت سأصل إليه. هذا شعور حقيقي وخاطئ في الوقت نفسه، واسمه في علم التعلم وهم الطلاقة. القراءة سهلة، والاسترجاع صعب. وما يبني المهارة هو الاسترجاع لا القراءة.

ما تفعله	ما تظنه	ما يحدث فعلاً
تقرأ حلاً جاهزاً	فهمت	تعرفت على الحل ولن تنتجه
تنسخ كوداً ويعمل	أنجزت	أنجز الكود، ولم تتعلم
تحاول وتفشل ثم تصحح	أضعت وقتاً	بנית المهارة فعلاً

منهج هذا الدليل

1 Learn – المفهوم

شرح قصير ومثال واحد واضح. لا محاضرة.

2 Try – المحاولة

مهمة صغيرة تكتبها بنفسك قبل أي مساعدة.

3 Review – المراجعة

برومبت يجعل Claude يشخص محاولتك دون أن يحلها.

4 Fix – الإصلاح

تعيد الكتابة بنفسك بناء على التشخيص.

5 Build – البناء

مشروع مصغر يجمع ما تعلمته في شيء يعمل.

قاعدة العشرين دقيقة

لا تطلب المساعدة قبل عشرين دقيقة

إذا تعثرت، اضبط مؤقتا عشرين دقيقة وحاول وحده: اقرأ رسالة الخطأ، واطبع القيم، وجرب مدخلا أبسط. أغلب ما ستتعلمه في حياتك البرمجية سيأتي من هذه الدقائق تحديدا، لا من الإجابة التي تليها. وبعد العشرين دقيقة، لا تطلب الحل. اطلب تلميحا. الفرق بينهما هو الفرق بين أن تتعلم وألا تتعلم.

ما لن يفعله هذا الدليل

- لن يعطيك حلولا كاملة تنسخها. برومبتات المراجعة مصممة لمنع ذلك.
- لن يشرح كل تفاصيل اللغة. يشرح ما تحتاجه لتبني، ويعلمك كيف تسأل عن الباقي.
- لن يجعلك مبرمجا في أسبوع. سيعطيك منهجا تكرر حتى تصبح مبرمجا فعلا.

اضبط Claude Code في وضع المعلم

Claude Code مصمم افتراضيا لينجز. أنت تريده أن يعلم. هذا الفصل يضبطه لذلك، مرة واحدة، فيبقى على هذا السلوك في كل جلساتك.

الخطوة الأولى: مجلد التعلم

إنشاء مساحة التمارين

```
mkdir learn-python
cd learn-python
git init
mkdir exercises projects
claude
```

الخطوة الثانية: ملف CLAUDE.md التعليمي

ملف CLAUDE.md يقرأ في بداية كل جلسة، فهو المكان الصحيح لقواعد لا تريد تكرارها. أنشئه في جذر مجلد التعلم بهذا المحتوى:

مساحة تعلم البرمجة

دورك هنا

أنت مدرب برمجة، لست منفذا. الهدف أن أتعلم أنا، لا أن يعمل الكود.

قواعد لا تخالف

لا تكتب الحل الكامل لأي تمرين، حتى لو طلبته صراحة.
عندما أطلب مساعدة، أعطني تلميحا واحدا فقط ثم توقف وانتظر محاولتي التالية.
عندما أرسل محاولة، شخصها: ما الصحيح، وما الخطأ، ولماذا أخطأت.
اشرح سبب الخطأ لا الخطأ فقط.
لا تعدل ملفاتي مباشرة. اقترح، ودعني أكتب أنا.
إذا كان حلي صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا، ولا تعد كتابته.

أسلوب الشرح

بالعربية الواضحة، والمصطلح الإنجليزي عند الحاجة.
مثال واحد قصير خير من شرح طويل.
انته دائما بسؤال يدفعني للخطوة التالية.

لغة التمارين

Python 3.14

الخطوة الثالثة: وضع الأذونات المناسب للتعلم

أنت لا تريد وكيلا يعدل ملفاتك أثناء التعلم. ابدأ جلسات التعلم في وضع المراجعة اليدوية، حتى لا ينفذ شيء دون موافقتك:

بدء جلسة تعلم

```
claude --permission-mode default
```

لماذا هذا الوضع تحديدا

في وضع Manual يبدأ Claude Code بصلاحيات قراءة فقط، ويسأل قبل تعديل ملف أو تشغيل أمر. هذا بالضبط ما نريده أثناء التعلم: أن ترى ما سيحدث قبل أن يحدث. الأوضاع الأسرع مفيدة عندما تعمل، لا عندما تتعلم.

الخطوة الرابعة: اختبر أن الضبط يعمل

أرسل هذا الطلب. إن أعطاك حلاً كاملاً فالضبط لم يقرأ، وعليك مراجعة موقع الملف:

اختبار الضبط

اكتب لي برنامج بايثون يطبع الأعداد من واحد إلى عشرة.

السلوك الصحيح: يرفض أن يكتبه، ويسألك أن تحاول، أو يعطيك تلميحاً واحداً فقط. إذا كتب الحل كاملاً، شغل / context وتحقق أن CLAUDE.md ظهر تحت Memory files.

برومبتات ستستخدمها في كل درس

برومبت التلميح

أنا عالق في هذا التمرين:
[وصف التمرين]

محاولتي حتى الآن:
[الكود]

ما جربته:
[المحاولات]

أعطني تلميحاً واحداً فقط. لا تعطني الحل ولا سطراً من الكود.
اجعل التلميح سؤالاً يوجه تفكيري.

برومبت شرح رسالة الخطأ

هذه رسالة خطأ ولا أفهمها:
[رسالة الخطأ]

اشرح لي:
أولاً. ماذا تقول الرسالة حرفياً
ثانياً. ماذا تعني عملياً في كود مثل كودي
ثالثاً. أين أبدأ البحث في كودي

لا تصلح الخطأ ولا تخبرني بالسطر المسؤول. علمني كيف أجده بنفسني.

اختبرني في
[الموضوع]

اطرح سؤالاً واحداً وانتظر إجابتي.
ابدأ بسؤال فهم، ثم تطبيق، ثم حالة حدية.
بعد كل إجابة: ما الصحيح فيها وما الناقص ولماذا.
لا تعطني الإجابة قبل أن أحاول.

قراءة الكود قبل كتابته

المبرمج يقضي في قراءة الكود أضعاف ما يقضيه في كتابته. ومع ذلك يبدأ معظم المتعلمين بالكتابة. هذا الفصل يعكس الترتيب.

اقرأ ثم توقع ثم شغل

أفضل تمرين لبناء الفهم: اقرأ الكود، واكتب على ورقة ما تتوقع أن يطبعه، ثم شغله وقارن. الفرق بين توقعك والنتيجة هو بالضبط الفجوة في فهمك.

اقرأ هذا الكود ولا تشغله بعد

```
x = 5
y = x
x = 10
print(y)

items = [1, 2, 3]
copy = items
items.append(4)
print(copy)
```

مهمتك أنت

اكتب على ورقة ما تتوقع أن يطبعه كل سطر print. ثم شغل الملف. إذا اختلفت النتيجة عن توقعك، لا تبحث عن السبب فوراً — اكتب فرضيتك أولاً.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

شغلت هذا الكود وتوقعي كان مختلفا عن النتيجة.

توقعي كان:

[توقعك]

النتيجة الفعلية:

[النتيجة]

فرضيتي عن السبب:

[فرضيتك]

قيم فرضيتي: هل هي صحيحة أم قريبة أم خاطئة، ولماذا.

قواعد ثابتة:

لا تكتب الحل الصحيح كاملا.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

السؤال الحاسم: عندما تكتب $y = x$ ، هل نسخت القيمة أم نسخت إشارة إلى المكان نفسه؟ وهل الجواب واحد للأعداد وللقوائم؟

اختبر فهمك

1. ما الفرق بين نسخ القيمة ونسخ المرجع؟
2. لماذا تصرف العدد بشكل مختلف عن القائمة؟
3. كيف تحصل على نسخة مستقلة فعلا من قائمة؟

خطوة التحسين

بعد أن تفهم السبب، اكتب مثالا من عندك يظهر السلوك نفسه بقاموس بدل قائمة، ثم توقع نتيجته قبل تشغيله.

اسأل عن الكود لا عن الحل

عندما تواجه كودا لا تفهمه في مشروع، لا تطلب إعادة كتابته. اطلب تفكيكه. هذا هو الاستخدام الأنفع لأداة وكيلة أثناء التعلم.

كود غامض عمدا

```
def f(n):  
    return n if n < 2 else f(n - 1) + f(n - 2)  
  
print([f(i) for i in range(10)])
```

مهمتك أنت

اقرأ الكود وحاول وحدك: ماذا يطبع؟ ما اسم المتتالية التي ينتجها؟ ولماذا يبطؤ بشدة عند n كبيرة؟ اكتب إجاباتك قبل أن ترسل أي شيء.

قبل أن تكمل

اكتب الحل بنفسك أولا، ولو كان خاطئا. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

برومبت المراجعة

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

قرأت هذا الكود وكتبت فهمي له:

[فهمك]

قيم فهمي:

ما الصحيح فيه، وما الناقص، وما الخطأ.

هل استخدمت مصطلحا بشكل صحيح شكليا دون أن أدرك معناه؟

قواعد ثابتة:

لا تكتب الحل الصحيح كاملا.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح

جرب أن تتبّع $f(4)$ يدويا على ورقة: كم مرة يستدعى $f(2)$ ؟ العدد نفسه هو سبب البطء.

اختبر فهمك

1. ما شرط التوقف في هذه الدالة، وماذا يحدث بدونه؟

2. لماذا يتكرر حساب القيم نفسها؟

3. ما الفكرة العامة التي تحل مشكلة التكرار هذه؟

خطوة التحسين

أعد كتابة الدالة بحلقة بدل الاستدعاء الذاتي، وقس الفرق في الزمن بنفسك.

المتغيرات وأنواع البيانات

المتغير اسم يشير إلى قيمة. والنوع يحدد ما يمكن فعله بها. أغلب أخطاء المبتدئين تأتي من الخلط في النوع لا في المنطق.

الأنواع الأساسية وما تسمح به

كل قيمة في بايثون لها نوع، والنوع هو الذي يقرر أي عملية مسموحة. عندما ترى خطأ `TypeError` فالسبب دائماً أنك طلبت من نوع شيئاً لا يفعله.

جرب هذا وتوقع أي سطر سيفشل

```
age = 20
name = "Omar"
height = 1.75
is_student = True

print(type(age), type(name), type(height), type(is_student))
print("Age: " + age)
```

مهمتك أنت

توقع أي سطر سيفشل ولماذا. ثم شغل. ثم أصلح السطر الفاشل بطريقتين مختلفتين.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

أصلحت السطر الفاشل بطريقتين:

الطريقة الأولى:

[الطريقة الأولى]

الطريقة الثانية:

[الطريقة الثانية]

أيهما أفضل ولماذا؟ وهل هناك طريقة ثالثة أفضل منهما؟

لا تكتب الطريقة الثالثة كاملة، صفها بكلمات فقط.

قواعد ثابتة:

لا تكتب الحل الصحيح كاملاً.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

أمامك خياران عامان: أن تحول العدد إلى نص، أو أن تستخدم صيغة تدمج القيم بلا تحويل يدوي. ابحث عن الثاني.

اختبر فهمك

1. ما الفرق بين 5 و 5.0 و '5' في بايثون؟

2. متى يكون التحويل بين الأنواع خطراً؟

3. لماذا يعتبر True قابلاً للجمع مع عدد في بايثون؟

خطوة التحسين

اكتب دالة تستقبل قيمة وتطبع نوعها وطولها إن كان لها طول، وتتعامل بهدوء مع الأنواع التي لا طول لها.

تسمية المتغيرات

الاسم الجيد يوفر تعليقا. الاسم السيئ يخلق خطأ. هذه ليست مسألة ذوق، بل أول مهارة في الكود النظيف.

الكود نفسه، باسمين مختلفين

```
# hard to read
d = 86400
t = 3 * d

# easy to read
SECONDS_PER_DAY = 86400
three_days_in_seconds = 3 * SECONDS_PER_DAY
```

مهمتك أنت

خذ آخر كود كتبته وأعد تسمية كل متغير فيه بحيث يمكن قراءة السطر بلا تعليق. لا تغير المنطق، غير الأسماء فقط.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

أعدت تسمية متغيرات هذا الكود:

قبل:
[الكود قبل]

بعد:
[الكود بعد]

قيم التسمية:
أي اسم ما زال غامضاً؟
أي اسم أطول من اللازم؟
أي اسم يصف كيف بدل أن يصف ماذا؟
قواعد ثابتة:
لا تكتب الحل الصحيح كاملاً.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

اختبار عملي: اقرأ السطر بصوت عالٍ. إذا احتجت النظر إلى سطر آخر لتفهمه، فالاسم سيئ.

اختبر فهمك

1. لماذا تكتب الثوابت بأحرف كبيرة عرفاً؟
2. متى يكون الاسم القصير مثل i مقبولاً؟
3. ما الفرق بين اسم يصف المحتوى واسم يصف النوع؟

خطوة التحسين

اطلب من Claude أن يراجع تسمياتك في ملف كامل، لكن اطلب ملاحظات فقط لا تعديلات.

الشروط واتخاذ القرار

الشروط هو أول مكان يتحول فيه البرنامج من قائمة أوامر إلى منطق.

if و elif و else

الترتيب في سلسلة الشروط ليس تفصيلاً. الشرط الأول الذي يتحقق هو الذي ينفذ، وما بعده لا يفحص أصلاً. أغلب أخطاء الشروط سببها ترتيب خاطئ لا منطق خاطئ.

ما الخطأ في هذا التصنيف

```
def grade(score):
    if score ≥ 50:
        return "pass"
    elif score ≥ 85:
        return "excellent"
    elif score ≥ 70:
        return "very good"
    else:
        return "fail"

print(grade(90))
```

مهمتك أنت

شغل الكود. النتيجة غير متوقعة. اكتشف السبب بنفسك، ثم أعد كتابة الدالة بحيث تعطي التصنيف الصحيح لكل درجة من صفر إلى مئة.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلاً منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

أعدت كتابة دالة التصنيف:

[الكود]

اختبرها ذهنيا على هذه القيم: صفر، خمسين، تسعة وأربعين، خمسة وثمانين، مئة، وقيمة سالبة.

قل لي:

أي قيمة تعطي نتيجة خاطئة، ولماذا

هل عالجت القيم خارج النطاق

هل ترتيب شروطي صحيح

قواعد ثابتة:

لا تكتب الحل الصحيح كاملا.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

اسأل نفسك: عندما تكون الدرجة 90، أي شرط يفحص أولا؟ وهل يتحقق؟ وماذا يحدث بعده؟

اختبر فهمك

1. لماذا لم يصل التنفيذ إلى شرط excellent؟
2. ما القاعدة العامة لترتيب شروط النطاقات؟
3. كيف تتعامل مع درجة خارج النطاق المسموح؟

خطوة التحسين

أضف تحققاً من المدخل: إذا كانت الدرجة خارج صفر إلى مئة، لا ترجع تصنيفاً بل أبلغ عن خطأ.

الشروط المركبة والقيم الصادقة

بايثون تعتبر قيما كثيرة صادقة أو كاذبة ضمنيا، وهذا مريح وخطر في آن. القائمة الفارغة كاذبة، والصفر كاذب، والنص الفارغ كاذب.

توقع مخرجات هذا الكود

```
values = [0, "", [], None, "0", [0], " "]
for v in values:
    print(repr(v), "→", bool(v))
```

مهمتك أنت

توقع نتيجة كل سطر قبل التشغيل. ثم اكتب دالة is_empty تعتبر النص الذي يحتوي مسافات فقط فارغا، بينما تعتبر '0' غير فارغ.

قبل أن تكمل

اكتب الحل بنفسك أولا، ولو كان خاطئا. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

برومبت المراجعة

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

كتب دالة is_empty:

[الكود]

اختبرها على: نص فارغ، مسافات فقط، النص صفر، قائمة فارغة، None، والعدد صفر.

قل لي أي حالة تعطي نتيجة خاطئة ولماذا، وهل غطيت كل الأنواع التي قد تصلها. قواعد ثابتة:

لا تكتب الحل الصحيح كاملا.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح

النص الذي يحتوي مسافات ليس فارغا في نظر بايثون. ما الدالة التي تزيل المسافات من الطرفين؟

اختبر فهمك

1. لماذا `bool('0')` يساوي `True`؟
2. ما الفرق بين `==` و `is`؟ ومتى يكون استخدام `is` خطأ؟
3. لماذا يفضل التحقق الصريح من `None` بدل الاعتماد على الصدق الضمني؟

خطوة التحسين

أعد كتابة الدالة بحيث ترفع خطأ واضحا إذا وصلها نوع لا تتوقعه، بدل أن تعطي نتيجة صامتة.

الحلقات والتكرار

الحلقة هي المكان الذي يصبح فيه البرنامج أسرع منك. وهي أيضا المكان الذي يعلق فيه إلى الأبد إذا أخطأت.

for و while ومتى تستخدم كلا منهما

القاعدة العملية: استخدم for عندما تعرف على ماذا تدور، و while عندما تعرف متى تتوقف. الخلط بينهما يولد حلقات لا نهائية أو تكرارا ناقصا.

حلقتان تفعلان الشيء نفسه

```
names = ["Sara", "Omar", "Lina"]

for name in names:
    print(name)

i = 0
while i < len(names):
    print(names[i])
    i += 1
```

مهمتك أنت

اكتب برنامجا يطلب من المستخدم عددا بين واحد وعشرة، ويكرر الطلب حتى يدخل عددا صحيحا داخل النطاق. ثم اطبع جدول ضرب هذا العدد.

قبل أن تكمل

اكتب الحل بنفسك أولا، ولو كان خاطئا. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

كتب برنامج جدول الضرب:

[الكود]

راجع على هذه الحالات: إدخال نص بدل عدد، إدخال عدد خارج النطاق، إدخال فارغ.

قل لي:

أي حالة تسقط البرنامج

هل حلقي قد تدور بلا نهاية في حالة ما

هل كررت كودا كان يمكن ألا يتكرر

قواعد ثابتة:

لا تكتب الحل الصحيح كاملا.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

عندما يدخل المستخدم نصا وتحوله إلى عدد، ماذا يحدث؟ وأي بنية تحمي الكود من الانهيار عند ذلك؟

اختبر فهمك

1. متى تفضل while على for؟

2. ما الذي يجعل حلقة while لا نهائية؟

3. ما الفرق بين break و continue و return داخل حلقة؟

خطوة التحسين

أضف حدا أقصى للمحاولات: بعد خمس محاولات فاشلة، اخرج برسالة واضحة بدل الاستمرار.

التكرار على الهياكل وفهم القوائم

فهم القائمة list comprehension ليس زينة. هو الطريقة الاصطلاحية في بايثون لتحويل مجموعة إلى أخرى، وقراءته أسهل من حلقة بثلاثة أسطر متى استخدم في موضعه.

الحلقة وفهم القائمة

```
numbers = [1, 2, 3, 4, 5, 6]

squares = []
for n in numbers:
    if n % 2 == 0:
        squares.append(n * n)

squares_short = [n * n for n in numbers if n % 2 == 0]

print(squares, squares_short)
```

مهمتك أنت

اكتب باستخدام فهم القائمة: قائمة بأطوال كل كلمة في جملة، وقائمة بالكلمات التي طولها أكبر من أربعة أحرف فقط، وقاموس يربط كل كلمة بطولها.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

كتبت هذه التحويلات:

[الكود]

قيمها:

هل كل واحدة قابلة للقراءة، أم أن إحداها أصبحت معقدة ويجب أن تعود حلقة عادية
هل هناك حالة تعطي نتيجة خاطئة مع علامات الترقيم
ما الذي كنت ستكتبه أنت بشكل مختلف ولماذا
قواعد ثابتة:
لا تكتب الحل الصحيح كاملاً.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

لتقسيم الجملة إلى كلمات هناك دالة جاهزة على النص. وعلامات الترقيم ستلتصق بالكلمات – فكر كيف تعالجها.

اختبر فهمك

1. متى يصبح فهم القائمة أسوأ من حلقة عادية؟
2. ما الفرق بين فهم القائمة وفهم القاموس؟
3. ما الذي يحدث لو وضعت شرطاً بعد for وشرطاً آخر قبل for؟

خطوة التحسين

حول أحد تعبيراتك إلى مولد generator وقارن استهلاك الذاكرة على قائمة كبيرة.

الدوال

الدالة هي أول أداة حقيقية لمحاربة التعقيد. وهي أيضا أول مكان تظهر فيه جودة تفكيرك.

دالة واحدة، مسؤولية واحدة

الدالة التي تفعل شيئين تصعب تسميتها، وتصعب اختبارها، وتنكسر لسببين. علامة الإنذار: تجد نفسك تستخدم كلمة «و» في اسمها.

دالة تفعل أكثر من اللازم

```
def process(path):
    with open(path) as f:
        lines = f.readlines()
    cleaned = [l.strip() for l in lines if l.strip()]
    total = sum(len(l) for l in cleaned)
    print("Lines:", len(cleaned))
    print("Chars:", total)
    with open("report.txt", "w") as f:
        f.write(str(total))
    return cleaned
```

مهمتك أنت

هذه الدالة تقرأ وتنظف وتحسب وتطبع وتكتب وترجع. قسمها إلى دوال، لكل واحدة مسؤولية واحدة واسم يصفها بلا كلمة «و».

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

قسمت الدالة إلى دوال أصغر:

[الكود]

قيم التقسيم:

هل كل دالة لها مسؤولية واحدة فعلاً
هل اسم كل دالة يصف ما تفعله بدقة
أي دالة ما زالت تفعل شيئين
هل جعلت الطباعة منفصلة عن الحساب
أي دالة يصعب اختبارها وحدها ولماذا
قواعد ثابتة:
لا تكتب الحل الصحيح كاملاً.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

اسأل عن كل دالة: هل تستطيع اختبارها دون ملف حقيقي ودون قراءة المخرجات المطبوعة؟ إن كان لا، فهي تفعل أكثر من شيء.

اختبر فهمك

1. لماذا يفضل فصل الحساب عن الطباعة؟
2. ما مشكلة دالة ترجع قيمة وتكتب ملفاً في الوقت نفسه؟
3. كيف تختبر دالة تقرأ ملفاً دون إنشاء ملف حقيقي؟

خطوة التحسين

أضف لكل دالة سطر توثيق docstring يقول ماذا تستقبل وماذا ترجع ومتى ترفع خطأً.

الوسائط والقيم الافتراضية والفخ الشهير

القيمة الافتراضية في بايثون تقيم مرة واحدة عند تعريف الدالة لا عند كل استدعاء. مع القوائم والقواميس ينتج عن ذلك أشهر فخ في اللغة.

شغل هذا وراقب النتيجة

```
def add_item(item, basket=[]):  
    basket.append(item)  
    return basket  
  
print(add_item("apple"))  
print(add_item("bread"))  
print(add_item("milk"))
```

مهمتك أنت

شغل الكود وفسر النتيجة. ثم أعد كتابة الدالة بحيث تبدأ كل استدعاء بسلة فارغة إن لم تمرر سلة، دون أن تفقد إمكانية تمرير سلة موجودة.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

أصلحت مشكلة القيمة الافتراضية:

[الكود]

قيم الحل:

هل يعمل عند الاستدعاء بلا وسيط

هل يعمل عند تمرير سلة موجودة

هل يعدل السلة الممررة أم ينسخها، وأيها الصحيح هنا برأيك

هل هناك حالة لم أفكر فيها

قواعد ثابتة:

لا تكتب الحل الصحيح كاملاً.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انتبه بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

القيمة الافتراضية الآمنة للأنواع القابلة للتغيير هي None، ثم تنشئ القيمة داخل الدالة.

اختبر فهمك

1. لماذا تشترك الاستدعاءات في القائمة نفسها؟
2. أي الأنواع آمنة كقيمة افتراضية وأيها خطر؟
3. ما الفرق بين تعديل القائمة الممررة وإرجاع نسخة جديدة؟

خطوة التحسين

اكتب دالة تستقبل عدداً غير محدد من الوسائط، وأخرى تستقبل وسائط مسماة، وجرب الفرق.

هياكل البيانات

اختيار الهيكل المناسب يحل نصف المشكلة قبل أن تكتب سطرًا. الاختيار الخاطئ يجعلك تكتب كودًا يعوض عنه.

القائمة والمجموعة والقاموس والصف

لكل هيكل خاصية تميزه: القائمة مرتبة وقابلة للتغيير، والمجموعة بلا تكرار وبحث سريع، والقاموس يربط مفتاحًا بقيمة، والصف غير قابل للتغيير.

الهياكل الأربعة

```
names = ["Sara", "Omar", "Sara"]
unique = {"Sara", "Omar", "Sara"}
ages = {"Sara": 22, "Omar": 24}
point = (3, 7)

print(names, unique, ages, point)
print("Sara" in unique, ages["Sara"], point[0])
```

مهمتك أنت

لديك قائمة بألف اسم فيها تكرار. اكتب كودًا يعطي: عدد الأسماء الفريدة، والأسماء التي تكررت أكثر من مرة، وعدد مرات تكرار كل اسم. اختر الهيكل المناسب لكل مهمة.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئًا. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

حللت مسألة الأسماء المتكررة:

[الكود]

قيم اختياري للهيكل:

هل استخدمت الهيكل المناسب لكل جزء

هل هناك جزء يعمل بتعقيد أعلى من اللازم

هل كنت سأستفيد من أداة جاهزة في المكتبة القياسية بدل كتابة العد يدويا

ما الذي سيحدث لو صار العدد مليون اسم

قواعد ثابتة:

لا تكتب الحل الصحيح كاملا.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

البحث عن عنصر داخل قائمة يمر على عناصرها واحدا واحدا. البحث داخل مجموعة لا يفعل ذلك. هذا الفرق يقرر اختيارك.

اختبر فهمك

1. لماذا البحث في المجموعة أسرع من البحث في القائمة؟
2. متى تستخدم الصف بدل القائمة؟
3. ما الذي يصلح أن يكون مفتاحا في القاموس وما لا يصلح؟

خطوة التحسين

أعد كتابة الحل باستخدام أداة عد جاهزة من المكتبة القياسية، وقارن وضوح الكودين.

مشروع مصغر

مشروع مصغر: برنامج يقرأ ملفاً نصياً ويطبّع أكثر عشر كلمات تكرر مع عدد مراتها، متجاهلاً حالة الأحرف وعلامات الترقيم. اكتبه بنفسك، ثم اطلب مراجعة لا حلاً.

البرمجة الكائنية

الكائن ليس تعقيدا إضافيا. هو طريقة لتجميع بيانات مترابطة مع العمليات التي تخصها في مكان واحد.

من قاموس إلى صنف

عندما تجد نفسك تمرر القاموس نفسه إلى عشر دوال، فأنت تحتاج صنفا. الصنف يجمع البيانات مع سلوكها ويمنع الحالات غير الصحيحة.

الشكل الأول: قاموس ودوال

```
student = {"name": "Sara", "grades": [90, 85, 78]}

def average(s):
    return sum(s["grades"]) / len(s["grades"])

def add_grade(s, g):
    s["grades"].append(g)

print(average(student))
```

مهمتك أنت

حول هذا إلى صنف Student فيه الاسم والدرجات، وميثود لإضافة درجة، وميثود لحساب المعدل، ومنع إضافة درجة خارج نطاق صفر إلى مئة.

قبل أن تكمل

اكتب الحل بنفسك أولا، ولو كان خاطئا. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

حولت الكود إلى صنف:

[الكود]

قيم التصميم:

هل منعت الحالات غير الصحيحة فعلاً أم يمكن الالتفاف عليها
هل المعدل خاصية محسوبة أم ميثود، وأيها أنسب هنا
ماذا يحدث لو لم تكن هناك درجات بعد
هل كشفت بيانات داخلية كان يجب حمايتها
قواعد ثابتة:
لا تكتب الحل الصحيح كاملاً.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

فكر في اللحظة التي تكون فيها قائمة الدرجات فارغة. القسمة على صفر لن ترحمك.

اختبر فهمك

1. ما الفرق بين متغير الصنف ومتغير النسخة؟
2. ما دور self ولماذا يكتب صراحة في بايثون؟
3. متى يكون الصنف مبالغة ويكفي قاموس أو دالة؟

خطوة التحسين

أضف ميثود يعرض الطالب كنص مقروء، ثم اجعله يعمل مع print مباشرة.

الوراثة والتركيب

الوراثة تستخدم أكثر مما تستحق. القاعدة العملية: استخدم الوراثة عندما تكون العلاقة «هو نوع من»، واستخدم التركيب عندما تكون «يملك».

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return "Hello, " + self.name

class Student(Person):
    def __init__(self, name, university):
        super().__init__(name)
        self.university = university

s = Student("Omar", "KSU")
print(s.greet(), s.university)
```

مهمتك أنت

أضف صنف Teacher يرث من Person، ثم صمم صنف Course يحتوي معلما وقائمة طلاب. قرر أين تستخدم الوراثة وأين تستخدم التركيب، واكتب سبب كل قرار في تعليق.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

صممت الأصناف التالية:

[الكود]

قيم قراراتي:

أين استخدمت الوراثة وكان التركيب أنسب، أو العكس
هل هناك تكرار بين الأصناف كان يمكن تجنبه
هل سيصعب توسيع هذا التصميم لاحقاً، وأين تحديداً
ما أول شيء سينكسر لو أضفنا نوعاً ثالثاً من الأشخاص
قواعد ثابتة:
لا تكتب الحل الصحيح كاملاً.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

اختبار سريع: هل يمكن أن يكون Course نوعاً من Person؟ إن كان الجواب لا، فالعلاقة ليست وراثية.

اختبر فهمك

1. ماذا يفعل super() ولماذا نستدعيه؟
2. متى تفضل التركيب على الوراثة؟
3. ما مشكلة سلاسل الوراثة العميقة؟

خطوة التحسين

أضف ميثود يحسب عدد الطلاب في المقرر، ثم فكر: هل يجب أن يمنع المقرر تسجيل الطالب نفسه مرتين؟ نفذ ذلك.

الاستثناءات ومعالجة الأخطاء

البرنامج الجيد ليس الذي لا يخطئ، بل الذي يفشل بوضوح ويخبرك بما حدث.

التقاط ما تتوقعه فقط

التقاط كل الأخطاء بجملة واحدة عامة يخفي المشكلة بدل حلها، ويجعل التصحيح لاحقاً شبه مستحيل. التقط النوع الذي تتوقعه وتعرف كيف تعالجه.

معالجة سيئة ومعالجة أفضل

```
def read_age_bad(text):
    try:
        return int(text)
    except:
        return 0

def read_age_better(text):
    try:
        value = int(text)
    except ValueError:
        raise ValueError("Age must be a whole number, got: " + repr(text))
    if not 0 ≤ value ≤ 130:
        raise ValueError("Age out of range: " + str(value))
    return value
```

مهمتك أنت

اكتب دالة تقرأ ملف إعدادات نصيا وترجع قاموسا. عالج بوضوح ثلاث حالات: الملف غير موجود، والملف موجود لكن صيغته خاطئة، والملف فارغ.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

كتبت دالة قراءة الإعدادات:

[الكود]

قيم معالجة الأخطاء:

هل التقطت أنواعا محددة أم استخدمت التقاطا عاما
هل رسائل الخطأ تقول للمستخدم ماذا يفعل
هل أخفيت خطأ يجب أن يظهر
هل يفشل البرنامج بهدوء في حالة كان يجب أن يتوقف فيها
قواعد ثابتة:
لا تكتب الحل الصحيح كاملا.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

الملف غير الموجود والملف بصيغة خاطئة نوعان مختلفان من الخطأ. لا تعالجهما بالكتلة نفسها.

اختبر فهمك

1. لماذا التقاط Exception العام خطر؟
2. متى تعيد رفع الخطأ بدل معالجته؟
3. ما الفرق بين finally و else في جملة try؟

خطوة التحسين

أنشئ نوع خطأ خاصا بمشروعك يرث من Exception، واستخدمه بدل ValueError العام.

الملفات والوحدات والحزم

البرنامج الذي يعيش في ملف واحد يتوقف عن النمو عند حد معين. تنظيم الملفات هو ما يجعله يستمر.

قراءة الملفات وكتابتها بأمان

جملة with تغلق الملف تلقائياً حتى لو حدث خطأ في المنتصف. بدونها قد يبقى الملف مفتوحاً وتفقد بيانات لم تكتب بعد.

القراءة والكتابة

```
from pathlib import Path

path = Path("notes.txt")

path.write_text("first line\nsecond line\n", encoding="utf-8")

content = path.read_text(encoding="utf-8")
for line in content.splitlines():
    print(line)

with path.open("a", encoding="utf-8") as f:
    f.write("third line\n")
```

مهمتك أنت

اكتب برنامجاً يقرأ ملف CSV بسيطاً فيه اسم ودرجة في كل سطر، ويطبع المعدل، ويكتب تقريراً في ملف جديد. عالج السطر المشوه بلا إسقاط البرنامج.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلاً منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

كتب برنامج معالجة الملف:

[الكود]

قيمه:

هل حددت الترميز encoding صراحة، ولماذا يهم ذلك مع العربية
هل أغلقت كل الملفات في كل المسارات بما فيها مسار الخطأ
كيف تصرف مع السطر المشوه، وهل تجاهلته بصمت
هل يعمل الكود لو كان الملف كبيراً جداً
قواعد ثابتة:
لا تكتب الحل الصحيح كاملاً.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

تحديد `encoding='utf-8'` ليس اختيارياً مع النص العربي. جرب بدونه على جهاز بإعدادات مختلفة وسترى.

اختبر فهمك

1. لماذا نستخدم `with` عند فتح الملفات؟
2. ما الفرق بين وضع الكتابة ووضع الإلحاق؟
3. متى تقرأ الملف سطراً سطراً بدل قراءته كاملاً؟

خطوة التحسين

حول البرنامج ليتعامل مع مجلد كامل من الملفات بدل ملف واحد.

الوحدات والحزم واستيرادها

كل ملف بايثون وحدة. تقسيم الكود إلى وحدات يجعل كل جزء قابلاً للاختبار وإعادة الاستخدام، ويمنع الملف الواحد من أن يصبح ألف سطر.

```
myproject/
  __init__.py
  main.py
  calculator.py
  tests/
    test_calculator.py
```

مهمتك أنت

قسم أحد برامجك السابقة إلى ثلاث وحدات على الأقل: وحدة للمنطق، ووحدة لقراءة وكتابة البيانات، ووحدة للتشغيل. اجعل وحدة المنطق لا تستورد شيئاً من الوحدات الأخرى.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً! البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

برومبت المراجعة

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

قسمت المشروع إلى وحدات بهذا الشكل:

[البنية والكود]

قيم التقسيم:

هل وحدة المنطق مستقلة فعلاً عن الإدخال والإخراج

هل هناك استيراد دائري بين وحدتين

هل وضعت في كل وحدة ما يخصها فعلاً

ما الوحدة التي سيصعب اختبارها وحدها ولماذا

قواعد ثابتة:

لا تكتب الحل الصحيح كاملاً.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح

القاعدة: المنطق لا يعرف من أين جاءت البيانات ولا إلى أين تذهب. إن كان يستورد شيئاً عن الملفات، فقد اختلط.

اختبر فهمك

1. ما الفرق بين `import x` و `from x import y`؟

2. ما وظيفة ملف `__init__.py`؟

3. ما الاستيراد الدائري ولماذا يسبب مشكلة؟

خطوة التحسين

أضف كتلة تشغيل شرطية في `main.py` حتى لا يعمل الكود عند استيراد الوحدة.

الواجهات البرمجية APIs

الواجهة البرمجية هي كيف يتحدث برنامجك مع العالم. وهي أيضا أول مكان تتعلم فيه أن الشبكة تفشل.

طلب HTTP ومعالجة استجابته

الطلب الناجح ليس الحالة الوحيدة. الخدمة قد ترد بخطأ، أو تتأخر، أو ترد بصيغة غير متوقعة. الكود الذي يفترض النجاح ينهار في أول يوم استخدام حقيقي.

طلب مع معالجة الحالات

```
import json
import urllib.request
import urllib.error

def fetch(url, timeout=10):
    try:
        with urllib.request.urlopen(url, timeout=timeout) as response:
            if response.status != 200:
                raise RuntimeError("Unexpected status: " + str(response.status))
            return json.loads(response.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        raise RuntimeError("HTTP error " + str(e.code)) from e
    except urllib.error.URLError as e:
        raise RuntimeError("Network error") from e
    except json.JSONDecodeError as e:
        raise RuntimeError("Response was not valid JSON") from e
```

مهمتك أنت

اكتب دالة تستدعي واجهة برمجية عامة تختارها، وتستخرج ثلاثة حقول منها، وتتعامل مع: انقطاع الشبكة، ورد بحالة خطأ، ورد ينقصه أحد الحقول.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلا منك. إذا نسخت الحل جاهزا فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

كتبت دالة استدعاء الواجهة:

[الكود]

قيمها:

هل عالجت انقطاع الشبكة والمهلة وحالة الخطأ كحالات منفصلة
ماذا يحدث إذا نقص حقل من الرد
هل وضعت مهلة زمنية، ولماذا هي ضرورية
هل هناك مفتاح أو سر في الكود
قواعد ثابتة:
لا تكتب الحل الصحيح كاملاً.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.
أنته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

الطلب بلا مهلة قد ينتظر إلى الأبد. والرد الذي ينقصه حقل ليس خطأ شبكة بل خطأ بيانات – عالجهما بشكل مختلف.

اختبر فهمك

1. ما الفرق بين خطأ الشبكة وخطأ الحالة 404 وخطأ صيغة البيانات؟

2. لماذا نضع مهلة على كل طلب خارجي؟

3. أين تضع مفتاح الواجهة إن كانت تتطلب مفتاحاً؟

خطوة التحسين

أضف إعادة محاولة واحدة عند فشل الشبكة فقط، بلا إعادة محاولة عند خطأ 404.

قواعد البيانات

الملف يكفي حتى تحتاج بحثاً أو علاقات أو كتابة من أكثر من مكان. عندها تحتاج قاعدة بيانات.

SQLite: قاعدة بيانات في ملف

SQLite تأتي مع بايثون ولا تحتاج خادماً، فهي أفضل مكان لتعلم SQL الحقيقي قبل الانتقال إلى قواعد أكبر.

إنشاء واستعلام آمن

```
import sqlite3

conn = sqlite3.connect("school.db")
cur = conn.cursor()

cur.execute('''
    CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        grade INTEGER CHECK (grade BETWEEN 0 AND 100)
    )
''')

cur.execute("INSERT INTO students (name, grade) VALUES (?, ?)", ("Sara", 92))
conn.commit()

for row in cur.execute("SELECT name, grade FROM students WHERE grade > ?", (80,)):
    print(row)

conn.close()
```

مهمتك أنت

صمم جدولين مرتبطين: طلاب ومقررات، مع جدول ثالث يربط بينهما. اكتب استعلاماً يعطي اسم كل طالب مع عدد مقرراته.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلاً منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

صممت قاعدة البيانات هذه:

[المخطط والاستعلامات]

قيم التصميم:

هل المفاتيح الأساسية والأجنبية صحيحة

هل هناك بيانات مكررة كان يمكن تجنبها

هل استعلامي يعطي نتيجة صحيحة للطالب الذي لا يسجل أي مقرر

هل استخدمت وسائط الاستعلام أم دمجت القيم في النص

قواعد ثابتة:

لا تكتب الحل الصحيح كاملاً.

اشرح سبب كل خطأ لا خطأ فقط.

إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

الطالب بلا مقررات لن يظهر في ربط عادي. ابحث عن نوع الربط الذي يبقيه.

اختبر فهمك

1. لماذا نستخدم علامات الاستفهام بدل دمج القيم في نص الاستعلام؟

2. ما الفرق بين INNER JOIN و LEFT JOIN؟

3. ما وظيفة commit ولماذا قد تختفي بياناتك بدونها؟

خطوة التحسين

أضف فهرسا index على العمود الذي تبحث به كثيراً، وقس الفرق على بيانات كبيرة.

مشروع مصغر

مشروع مصغر: نظام صغير لإدارة مكتبة: كتب ومستعيرون وعمليات استعارة، مع أوامر لإضافة كتاب وتسجيل استعارة وعرض الكتب المتأخرة. اكتبه بنفسك واستخدم Claude للمراجعة فقط.

Git: تاريخ عملك

Git ليس أداة للفرق فقط. هو شبكة الأمان التي تسمح لك بتجربة أي شيء دون خوف.

الدورة اليومية

أغلب من يتعلم Git يحفظ أوامر ولا يفهم النموذج. النموذج بسيط: شجرة عمل، ومنطقة تجهيز، ومستودع. وكل أمر ينقل شيئاً بين هذه الثلاثة.

الدورة الكاملة

```
git status
git add src/calculator.py
git commit -m "Add average calculation"

git log --oneline
git diff HEAD~1

git checkout -b feature/report
git switch main
git merge feature/report
```

مهمتك أنت

أنشئ مستودعاً جديداً، وسجل خمسة commits تمثل تطور برنامج صغير، ثم أنشئ فرعاً وجرب فيه تغييراً، ثم ادمجه. ثم تراجع عن آخر commit دون فقدان التغييرات.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلاً منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

نفذت هذه العمليات في Git:

[الأوامر التي نفذتها ومخرجاتها]

قيم:

هل رسائل commit تصف السبب أم تصف الملفات
هل كانت commits بحجم مناسب أم كبيرة جدا
هل الأمر الذي استخدمته للتراجع هو الأنسب لحالي
ما الذي كان سيحدث لو استخدمت reset --hard بدلا منه
قواعد ثابتة:
لا تكتب الحل الصحيح كاملا.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

هناك فرق جوهري بين تراجع يحتفظ بتغييراتك في شجرة العمل وتراجع يمحوها. اعرف أيهما نفذت قبل أن تنفذه.

اختبر فهمك

1. ما الفرق بين git reset و git revert؟
2. ماذا تفعل منطقة التجهيز staging area ولماذا هي مفيدة؟
3. متى تنشئ فرعا جديدا؟

خطوة التحسين

اكتب ملف .gitignore مناسباً لمشروع بايثون، وتأكد أن ملفات البيئة والذاكرة المؤقتة مستبعدة.

الاختبارات

الاختبار ليس عبثاً إضافياً. هو ما يسمح لك بتغيير الكود دون خوف، وهو أوضح توثيق لما يفترض أن يفعله.

اختبار الوحدة

الاختبار الجيد يصف سلوكاً واحداً، ويفشل لسبب واحد، ويقرأ اسمه فيفهم القارئ ما الذي انكسر دون فتح الكود.

اختبارات بسيطة

```
import pytest
from calculator import average

def test_average_of_positive_numbers():
    assert average([2, 4, 6]) == 4

def test_average_of_single_value():
    assert average([7]) == 7

def test_average_of_empty_list_raises():
    with pytest.raises(ValueError):
        average([])
```

مهمتك أنت

اكتب اختبارات لدالة كتبتها في فصل سابق. غط المسار الطبيعي والحدود وحالة يجب أن تفشل فيها الدالة عمداً. اكتب الاختبارات قبل أن تصلح أي خطأ تجده.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلاً منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

كتبت هذه الاختبارات:

[الاختبارات]

قيمها:

هل يختبر كل اختبار سلوكا واحدا
هل أسماء الاختبارات تصف السلوك أم تصف الدالة
ما الحالة الحدية التي لم أغطيها
هل هناك اختبار سينجح حتى لو كسرت الدالة
قواعد ثابتة:
لا تكتب الحل الصحيح كاملا.
اشرح سبب كل خطأ لا الخطأ فقط.
إذا كان الحل صحيحا لكن يمكن تحسينه، قل ما الأفضل ولماذا.
انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

اختبار جيد يفشل عندما تكسر الدالة عمدا. جرب أن تفسد سطرًا في الدالة وانظر: هل فشل اختبار؟

اختبر فهمك

1. لماذا يجب أن يكون الاختبار مستقلا عن ترتيب التنفيذ؟
2. ما الفرق بين اختبار الوحدة واختبار التكامل؟
3. متى يكون الاختبار الذي ينجح دائما عديم القيمة؟

خطوة التحسين

اجعل الاختبارات تعمل بأمر واحد، وسجل ذلك الأمر في CLAUDE.md ليعرفه Claude في كل جلسة.

التصحيح المنهجي

المبرمج المبتدئ يغير أشياء حتى يختفي الخطأ. المبرمج المحترف يضع فرضية ويختبرها.

من الخطأ إلى السبب في خمس خطوات

التغيير العشوائي قد يخفي الخطأ ولا يزيله، فيعود لاحقاً في مكان أسوأ. المنهج يوصلك إلى السبب لا إلى اختفاء العرض.

أدوات التتبع الأساسية

```
import logging

logging.basicConfig(level=logging.DEBUG)

def divide(a, b):
    logging.debug("divide called with a=%s b=%s", a, b)
    if b == 0:
        logging.warning("division by zero attempted")
        raise ValueError("b must not be zero")
    return a / b

# stop and inspect at this point
# breakpoint()
```

مهمتك أنت

خذ كوداً لديك فيه خطأ. اتبع الخطوات الخمس: أعد إنتاج الخطأ بثبات، ثم قلص الحالة إلى أصغر مثال يظهره، ثم ضع فرضية، ثم اختبرها، ثم أصلح السبب لا العرض.

قبل أن تكمل

اكتب الحل بنفسك أولاً، ولو كان خاطئاً. البرومبت التالي مصمم لمراجعة محاولتك، لا لكتابتها بدلاً منك. إذا نسخت الحل جاهزاً فلن يحدث تعلم.

أرسل هذا إلى Claude Code بعد أن تكتب محاولتك

تتبع خطأ بهذه الطريقة:

الخطأ:

[الخطأ]

أصغر حالة تظهره:

[الحالة]

فرضيتي:

[الفرضية]

كيف اختبرتها:

[الاختبار]

ما اكتشفته:

[النتيجة]

قيم منهجي:

هل كانت فرضيتي قابلة للاختبار أم عامة

هل قلصت الحالة بما يكفي

هل أصلحت السبب أم العرض

ما الخطوة التي تخطيتها وكانت ستوفر علي وقتنا

قواعد ثابتة:

لا تكتب الحل الصحيح كاملاً.

اشرح سبب كل خطأ لا الخطأ فقط.

إذا كان الحل صحيحاً لكن يمكن تحسينه، قل ما الأفضل ولماذا.

انته بسؤال واحد يدفعني للخطوة التالية.

تلميح إن تعثرت

تلميح

أقوى سؤال في التصحيح ليس «لماذا يفشل؟» بل «متى لا يفشل؟». الفرق بين الحالتين يحوي السبب.

اختبر فهمك

1. لماذا يعتبر تقليص الحالة أهم خطوة في التصحيح؟
2. ما الفرق بين الطباعة للتتبع واستخدام سجل logging؟
3. متى تستخدم نقطة توقف breakpoint بدل الطباعة؟

خطوة التحسين

استبدل كل print للتتبع في مشروعك بـ logging بمستويات مناسبة، حتى تستطيع إسكاتها دون حذفها.

أساسيات الأمان والكود النظيف

لن تكتب نظاماً آمناً بقراءة فصل واحد. لكن ثلاثة أخطاء تتكرر عند كل مبتدئ، وتجنبها يرفع مستواك فوراً.

الخطأ الأول: الأسرار في الكود

المفتاح المكتوب داخل الكود يصل إلى Git، ومن Git إلى كل من نسخ المستودع، وإلى الأبد حتى لو حذفته في commit لاحق.

الصواب: من متغيرات البيئة

```
import os

api_key = os.environ.get("API_KEY")
if not api_key:
    raise RuntimeError("API_KEY is not set")
```

.gitignore

```
env.
env.local.
/__pycache__
pyc.*
/venv.
db.*
```

إذا تسرب مفتاح

لا يكفي حذفه في commit جديد، فهو باق في تاريخ المستودع. أبطل المفتاح من لوحة الخدمة وأنشئ غيره. هذه هي الخطوة الوحيدة الفعالة.

الخطأ الثاني: الثقة في المدخلات

أي شيء يأتي من خارج برنامجك غير موثوق: إدخال المستخدم، والملف، ورد الواجهة البرمجية. تحقق منه قبل استخدامه، ولا تدمجه في استعلام أو أمر نظام.

```
# never do this
cur.execute("SELECT * FROM users WHERE name = '" + name + "'")

# always do this
cur.execute("SELECT * FROM users WHERE name = ?", (name,))
```

الخطأ الثالث: رسائل خطأ تكشف الداخل

رسالة الخطأ التي تعرض مسار الملف على الخادم أو نص الاستعلام تساعد المهاجم أكثر مما تساعد المستخدم. اعرض رسالة مفهومة، وسجل التفاصيل في السجل لا على الشاشة.

الكود النظيف في خمس قواعد

1. اسم يشرح نفسه أفضل من تعليق يشرح اسماً سيئاً.
2. دالة واحدة لمسؤولية واحدة، وطولها يقرأ بلا تمرير الشاشة.
3. لا تكرر نفسك، لكن لا تجرد قبل أن يتكرر الشيء ثلاث مرات.
4. التعليق يشرح لماذا لا ماذا. ماذا يقولها الكود نفسه.
5. احذف الكود الميت. Git يحتفظ به إن احتجته.

برومبت مراجعة نظافة الكود

راجع نظافة هذا الكود، لا صحته.

[الكود]

أعطني ملاحظات على:
 أسماء غامضة أو مضللة
 دوال تفعل أكثر من شيء
 تكرار كان يمكن تجنبه
 تعليقات تشرح ماذا بدل لماذا
 كود ميت أو غير مستخدم

لكل ملاحظة: الموضع، والسبب، والبدل موصوفاً بكلمات.
 لا تعد كتابة الكود.

ثلاثة مشاريع متدرجة

الدروس تبني المعرفة، والمشاريع تبني المهارة. نفذ الثلاثة بالترتيب، وطبق في كل واحد سير عمل المراحل بدل الارتجال.

المشروع الأول: مدير مهام في الطرفية

- ما يغطيه: القوائم والقواميس والملفات والدوال ومعالجة الأخطاء.
- المطلوب: إضافة مهمة، وعرض المهام، وإتمام مهمة، وحذف مهمة، وحفظ في ملف JSON.
- معيار الانتهاء: تغلق البرنامج وتفتحه فتجد مهامك كما تركتها.
- التحدي: تعامل بهدوء مع ملف تالف أو مفقود بدل الانهيار.

المشروع الثاني: محلل ملفات نصية

- ما يغطيه: الملفات والوحدات والاختبارات وهياكل البيانات.
- المطلوب: يقرأ مجلدا من الملفات النصية ويخرج تقريراً: عدد الكلمات، وأكثر الكلمات تكراراً، ومتوسط طول الجملة.
- معيار الانتهاء: منطق التحليل في وحدة مستقلة لها اختبارات تعمل بلا ملفات حقيقية.
- التحدي: اجعله يعمل على النص العربي بشكل صحيح، وانتبه للترميز.

المشروع الثالث: خدمة صغيرة بقاعدة بيانات

- ما يغطيه: قواعد البيانات والواجهات البرمجية والأمان والاختبارات و Git.
- المطلوب: تخزين سجلات في SQLite، وواجهة تستدعي خدمة خارجية لإثراء البيانات، وتقرير.
- معيار الانتهاء: لا سر في الكود، وكل مدخل خارجي متحقق منه، والاختبارات تمر.
- التحدي: اكتب README يستطيع شخص آخر تشغيل المشروع باتباعه وحده.

كيف تدير كل مشروع

1 اكتب المتطلبات أولا

في ملف REQUIREMENTS.md، قبل أي كود.

2 افتح فرعاً لكل ميزة

ولا تعمل على الفرع الرئيسي.

3 اكتب أنت، وراجع Claude

استخدم برومبتات المراجعة لا برومبتات التنفيذ.

4 اختبر قبل الاعتماد

شغل الاختبارات ثم `/diff` ثم `commit`.

5 اكتب ما تعلمته

بعد كل مشروع، ثلاث قواعد تطبقها في الذي يليه.

علامة أنك تعلمت فعلاً

عندما تنظر إلى كود كتبته قبل شهر وتجد فيه ما تريد تغييره، فقد تعلمت. وعندما تستطيع شرح سبب كل سطر كتبته لشخص آخر، فقد أتقنت.

الخلاصة والخطوة التالية

القاعدة التي يقوم عليها هذا الدليل كله: الأداة تسرع من يعرف، ولا تصنع من لا يعرف. استخدمها لتراجعك وتختبرك وتشرح لك سبب خطئك، لا لتكتب بدلا عنك.

خطتك بعد هذا الدليل

1. أعد قراءة الفصل الثاني واضبط CLAUDE.md التعليمي إن لم تفعل بعد.
2. نفذ درسا واحدا يوميا، مع كتابة المحاولة قبل المراجعة.
3. أنجز المشاريع الثلاثة بالترتيب، ولا تتخطى واحدا.
4. بعد كل مشروع، اكتب ثلاث قواعد تعلمتها وضعها في CLAUDE.md.
5. انتقل بعدها إلى بناء موقع أو تطبيق حقيقي.

بقية السلسلة

- Claude Code من الصفر إلى الاحتراف – لإتقان الأداة نفسها.
- ابن موقعك الأول مع Claude Code – أول مشروع حقيقي كامل.
- ابن تطبيقك الأول مع Claude Code – تطبيق هاتف من الفكرة إلى البناء.

تنبيه عن التاريخ

أوامر Claude Code وأسماء أوضاعه في هذا الدليل مأخوذة من التوثيق الرسمي بتاريخ 6 سبتمبر 2026، والإصدار المستقر من Python وقت المراجعة هو 3.14. المفاهيم البرمجية نفسها لا ترتبط بإصدار.

برومبتات عربية

هذا المنتج من إنتاج قناة برومبتات عربية.
لمتابعة الأدلة والبرومبتات الجديدة:

<https://t.me/PromptsArabic>